

WebRTC One-to-One Video Call

Phase 1 — Backend & Signaling Foundation

Issue #1: Backend Setup & API

Day 1 Sprint Documentation

Assignee: Aabhishek23

Repository: Aabhishek23/WEBRTC

Labels: enhancement · backend · setup · api · Day-1

June 29, 2026

Contents

1	Issue Overview	3
1.1	Objective	3
1.2	Scope	3
2	Project Structure	3
2.1	Directory Layout	3
2.2	Design Rationale	4
3	API Specification	4
3.1	POST /api/rooms	4
3.1.1	Success Response	4
3.1.2	Error Response	4
3.2	GET /api/health	5
3.2.1	Success Response	5
3.3	GET /api/rooms/:roomId	5
4	Implementation Details	6
4.1	Room ID Generation Algorithm	6
4.1.1	Algorithm Properties	6
4.2	Server Entry Point	6
4.3	Environment Configuration	7
5	Request–Response Flow Diagram	8
6	Testing & Verification	8
6.1	Manual Test Results	8
6.1.1	Test 1 — Health Check Endpoint	8
6.1.2	Test 2 — Room Creation	9
6.2	Acceptance Criteria Verification	9
7	npm Scripts Reference	10
7.1	Installing Dependencies	10
7.2	Installed Packages	10
8	Known Limitations & Future Work	10
8.1	Current Limitations (Phase 1)	10
8.2	Phase 2 / Phase 3 Improvements	10
9	GitHub Closing Comment Template	11
10	Revision History	11

1 Issue Overview

Issue ID: ISSUE-001

Day: 1

Type: Feature

Priority: High

Status: Completed

Estimated Hours: 4 hrs

Milestone: M1 — Server Foundation (Day 1-2)

1.1 Objective

This issue covers the complete setup of the **Node.js + Express.js** backend server for the WebRTC One-to-One Video Call application (Phase 1). The goal is to establish a working HTTP server with:

- Proper environment configuration via `dotenv`
- A **room creation API** (POST `/api/rooms`) returning a unique `XXX-XXX-XXX` room ID
- A **health check** endpoint (GET `/api/health`)
- Modular folder structure for scalability in Phase 2

1.2 Scope

Component	Technology	Purpose
HTTP Server	Node.js + Express.js	API request handling
Environment Config	dotenv	Secure config management
Room ID Generator	Custom utility	Unique XXX-XXX-XXX format IDs
Error Handling	Express middleware	Structured JSON error responses
Dev Server	nodemon	Auto-restart on file change

Table 1: Technology Stack for Issue #1

2 Project Structure

2.1 Directory Layout

The backend code resides inside the `server/` folder at the root of the repository. The folder structure implemented during this issue is:

```
1 WEBRTC/
2   server/
3     middleware/
4       errorHandler.js      # Global error handling middleware
5     routes/
6       rooms.js             # POST /api/rooms + GET /api/rooms/:id
7     utils/
8       roomIdGenerator.js   # XXX-XXX-XXX room ID generator
9     .env                   # Local environment variables (git-ignored)
10    .env.example            # Template for environment setup
11    package.json            # Project metadata & scripts
12    server.js               # Main Express server entry point
```

Listing 1: Backend Folder Structure

2.2 Design Rationale

The folder structure follows the **Separation of Concerns** principle:

- **Routes** contain only HTTP handler logic — no business logic.
- **Utils** hold reusable pure functions (e.g., ID generation).
- **Middleware** captures cross-cutting concerns (errors, logging).
- **server.js** is the composition root — it wires everything together.

This structure is forward-compatible with **Day 2 (Socket.IO)** and **Day 3 (Signaling)** additions without requiring refactoring.

3 API Specification

3.1 POST /api/rooms

Creates a new room and returns a unique 9-digit room ID.

Field	Value
Method	POST
Endpoint	/api/rooms
Authentication	None (Phase 1)
Request Body	Empty
Content-Type	application/json
Success Status Code	201 Created

Table 2: POST /api/rooms Endpoint Specification

3.1.1 Success Response

```
1 {  
2   "success": true,  
3   "roomId": "346-661-586",  
4   "message": "Room created successfully",  
5   "createdAt": "2026-06-28T18:30:03.418Z"  
6 }
```

Listing 2: POST /api/rooms — 201 Created Response

3.1.2 Error Response

```
1 {  
2   "success": false,  
3   "message": "Internal Server Error",  
4   "stack": "Error: ... (only in development mode)"  
5 }
```

Listing 3: POST /api/rooms — 500 Internal Server Error

Field	Value
Method	GET
Endpoint	/api/health
Success Status Code	200 OK

Table 3: GET /api/health Endpoint Specification

3.2 GET /api/health

Server liveness check — returns current environment and timestamp.

3.2.1 Success Response

```
1 {  
2   "status": "OK",  
3   "environment": "development",  
4   "timestamp": "2026-06-28T18:29:24.094Z"  
5 }
```

Listing 4: GET /api/health — 200 OK Response

3.3 GET /api/rooms/:roomId

Checks whether a given room ID is currently active.

```
1 {  
2   "success": true,  
3   "roomId": "346-661-586",  
4   "message": "Room exists"  
5 }
```

Listing 5: GET /api/rooms/346-661-586 — 200 OK

```
1 {  
2   "success": false,  
3   "message": "Room 000-000-000 not found"  
4 }
```

Listing 6: GET /api/rooms/000-000-000 — 404 Not Found

4 Implementation Details

4.1 Room ID Generation Algorithm

The unique room ID generator is the core business logic of this issue. The algorithm uses an **in-memory Set** to guarantee no duplicate IDs are issued during the server's lifetime.

```
1 const activeRooms = new Set();
2
3 const generateRoomId = () => {
4   let roomId;
5   do {
6     // Generate 9 random digits
7     const digits = Math.floor(
8       100000000 + Math.random() * 900000000
9     ).toString();
10    // Format as XXX-XXX-XXX
11    roomId = `${digits.slice(0,3)}-${digits.slice(3,6)}-${digits.slice(
12      6,9)}';
13  } while (activeRooms.has(roomId)); // Retry if duplicate
14
15  activeRooms.add(roomId);
16  return roomId;
17 };
```

Listing 7: roomIdGenerator.js — Core Algorithm

4.1.1 Algorithm Properties

- **Format:** XXX-XXX-XXX where each X is a digit (0–9)
- **Search Space:** $9 \times 10^8 = 900,000,000$ possible room IDs
- **Collision Probability (first call):** ≈ 0 (practically zero for Phase 1 scale)
- **Duplicate Prevention:** do-while loop with `Set.has()` check
- **Time Complexity:** $O(1)$ average case

4.2 Server Entry Point

The `server.js` file is the application's composition root. It:

1. Loads environment variables via `dotenv.config()`
2. Attaches JSON body parser middleware
3. Registers the `/api/rooms` route
4. Registers the `/api/health` inline route
5. Attaches the global error handler as the last middleware
6. Starts listening on the configured `PORT`

```
1 dotenv.config();
2 app.use(express.json());           // Parse JSON bodies
3 app.use('/api/rooms', roomRoutes); // Room management routes
4 app.get('/api/health', handler);   // Health check
5 app.use(errorHandler);             // Must be LAST middleware
6 app.listen(PORT, callback);
```

Listing 8: server.js – Middleware Pipeline

Important: The `errorHandler` middleware **must** be registered after all routes. Express identifies error-handling middleware by its 4-parameter signature: `(err, req, res, next)`.

4.3 Environment Configuration

```
1 # Server Environment Configuration
2 # Copy this file to .env and fill in your values
3
4 PORT=5000
5 NODE_ENV=development
```

Listing 9: .env.example — Environment Template

- `.env` is listed in `.gitignore` to prevent accidental credential commits.
- `.env.example` is committed as a reference template for new developers.
- `process.env.PORT || 5000` fallback ensures the server works even without `.env`.

5 Request–Response Flow Diagram

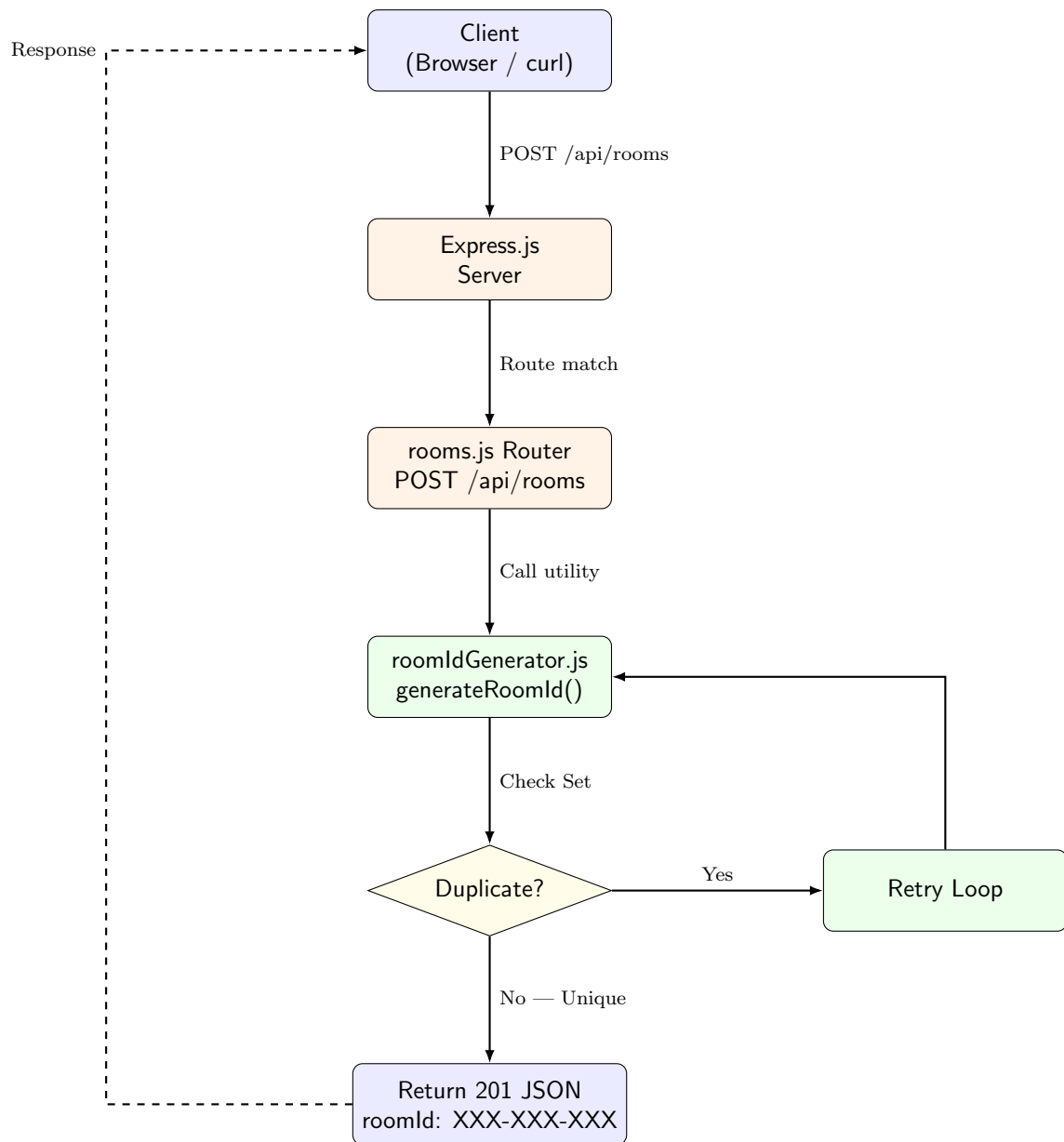


Figure 1: POST /api/rooms — Complete Request–Response Flow

6 Testing & Verification

6.1 Manual Test Results

All acceptance criteria were verified using `curl` against the running server at `http://localhost:5000`.

6.1.1 Test 1 — Health Check Endpoint

```
1 curl -s http://localhost:5000/api/health
```

Listing 10: Health Check curl Command


```
1 {
2   "status": "OK",
3   "environment": "development",
4   "timestamp": "2026-06-28T18:29:24.094Z"
5 }
```

Listing 11: Health Check — Actual Output

Result: **PASS** ✓

6.1.2 Test 2 — Room Creation

```
1 curl -s -X POST http://localhost:5000/api/rooms \
2   -H "Content-Type: application/json"
```

Listing 12: Room Creation curl Command

```
1 {
2   "success": true,
3   "roomId": "346-661-586",
4   "message": "Room created successfully",
5   "createdAt": "2026-06-28T18:30:03.418Z"
6 }
```

Listing 13: Room Creation — Actual Output

Result: **PASS** ✓ — Room ID format 346-661-586 matches XXX-XXX-XXX

6.2 Acceptance Criteria Verification

Acceptance Criterion	Expected	Status
Server starts on configured PORT	PORT 5000	PASS
POST /api/rooms returns XXX-XXX-XXX format ID	346-661-586	PASS
Environment variables load from .env	NODE_ENV=dev	PASS
Duplicate room IDs not generated	Set prevents dup	PASS
GET /api/health returns 200 OK	status: OK	PASS

Table 4: Acceptance Criteria Verification Results

All 5 acceptance criteria verified and passed.
Issue #1 is complete and ready for merge into main.

7 npm Scripts Reference

Script	Command	Use Case
npm run dev	nodemon server.js	Development (auto-restart)
npm start	node server.js	Production server start
npm test	Echo placeholder	Placeholder (Day 7 tests)

Table 5: Available npm Scripts

7.1 Installing Dependencies

```
1 # Navigate to server folder
2 cd server/
3
4 # Install production dependencies
5 npm install express dotenv uuid
6
7 # Install development dependency
8 npm install --save-dev nodemon
```

Listing 14: Dependency Installation

7.2 Installed Packages

Package	Version	Type	Purpose
express	^4.x	Production	HTTP server framework
dotenv	^16.x	Production	Environment variable loader
uuid	^9.x	Production	Available for future use
nodemon	^3.x	Development	Auto-restart dev server

Table 6: Installed npm Packages

8 Known Limitations & Future Work

8.1 Current Limitations (Phase 1)

- **In-Memory Storage:** Room IDs are stored in a **Set** in RAM. Server restart clears all active rooms. This is acceptable for Phase 1.
- **No Authentication:** POST /api/rooms has no rate limiting or authentication. Any client can create unlimited rooms.
- **No Room Expiry:** Rooms are never automatically cleaned from the **Set** unless a disconnect event fires (added in Day 2).

8.2 Phase 2 / Phase 3 Improvements

- Replace in-memory **Set** with **MySQL** database (Phase 3).
- Add **JWT authentication** to room creation (Phase 3).

- Add **rate limiting** middleware (e.g., `express-rate-limit`).
- Add **room expiry TTL** (Time-to-Live) logic.
- Generate **alphanumeric** room IDs for increased entropy.

9 GitHub Closing Comment Template

The following comment should be posted on Issue #1 in GitHub to officially close it:

```
1 Day 1 Complete!
2
3 - Express server running on PORT 5000
4 - POST /api/rooms -> returns unique XXX-XXX-XXX room ID (tested:
   346-661-586)
5 - GET /api/health -> server status check (200 OK confirmed)
6 - dotenv configured, .env.example added for team setup
7 - No duplicate room IDs (in-memory Set with do-while loop)
8 - nodemon configured for dev hot-reload (npm run dev)
9
10 All 5 acceptance criteria PASSED.
11
12 closes #1
```

Listing 15: GitHub Closing Comment for Issue #1

10 Revision History

Version	Date	Author	Changes
1.0	June 29, 2026	Aabhishek23	Initial documentation for Issue #1 completion

Table 7: Document Revision History